

Analisi e Confronto di Algoritmi di Ricerca su un Dataset Reale

Progetto a cura di:

Mazza	Matteo Emanuele	N46006811
Pagliarulo	Luca	N46006832
Marciano	Matteo	N46006830

Obiettivo del Progetto

Il progetto ha l'obiettivo di analizzare e confrontare le prestazioni di vari algoritmi di ricerca del cammino su un dataset reale rappresentato dalla rete di coacquisti di Amazon, aggiornata a marzo 2002. Il dataset viene utilizzato per testare algoritmi di ricerca classici, evidenziando i tempi di esecuzione, l'uso della memoria e la complessità operativa per ciascun approccio.

Dataset Utilizzato

- **Nome del file:** [Amazon0302.txt](#)
- **Descrizione:** Il dataset contiene un grafo diretto che rappresenta le vendite su Amazon. Ogni nodo è un prodotto e ogni arco diretto rappresenta la coacquisto di un prodotto con un altro.
 - **Nodo:** rappresenta un prodotto.
 - **Arco diretto:** rappresenta la relazione "i clienti che hanno acquistato questo articolo hanno acquistato anche". Se un prodotto i è frequentemente acquistato insieme al prodotto j, il grafo contiene un arco diretto da i a j.

Dataset Statistics	
Nodes	262111
Edges	1234877
Nodes in largest WCC	262111 (1.000)
Edges in largest WCC	1234877 (1.000)
Nodes in largest SCC	241761 (0.922)
Edges in largest SCC	1131217 (0.916)
Average clustering coefficient	0.4198
Number of triangles	717719
Fraction of closed traingles	0.09339
Diameter (longest shortest path)	32
90-percentile effective diameter	11

Algoritmi di Ricerca Implementati

1. Breadth-First Search (BFS):

- Ricerca in ampiezza.
- Esplora tutti i nodi adiacenti prima di procedere a un livello di profondità successivo.
- Garanzia di trovare il percorso più corto in un grafo non pesato.

2. Depth-First Search (DFS):

- Ricerca in profondità.
- Esplora ogni ramo fino alla sua fine (dunque al raggiungimento di una foglia) prima di tornare indietro.
- Potrebbe non trovare il percorso ottimale ma è più adatto per esplorazioni complete.

3. A*:

- Ricerca euristica che combina il costo del cammino già percorso e una stima del costo rimanente.
- Utilizza una funzione euristica per trovare il cammino ottimale.
- Adatto per grafi pesati.

4. Uniform Cost Search (UCS):

- Algoritmo di Dijkstra
- Variante della BFS che funziona sui grafi con pesi, esplorando sempre il percorso con il costo accumulato più basso.
- Trova il percorso a costo minimo.

5. Greedy Search:

- Utilizza una funzione euristica per guidare la ricerca verso l'obiettivo, ma non garantisce il percorso ottimale.
 - Si concentra sulla prossimità all'obiettivo senza considerare il costo complessivo.
-

Funzionalità Principali

1. Carica_file_grafo

- Carica il file di testo .txt contenente i dati del grafo e lo converte in una struttura dati (lista di adiacenza).

2. Parse_grafo

- Legge il file e costruisce il grafo sotto forma di dizionario, dove ogni chiave è un nodo e il valore associato è una lista di nodi adiacenti.

3. Traccia_percorso

- Ricostruisce il percorso dall'inizio all'obiettivo utilizzando un dizionario di genitori generato durante l'esecuzione degli algoritmi di ricerca.

4. Esegui_Ricerca_AlgoritmoX

- Esegue uno specifico algoritmo di ricerca (BFS, DFS, A*, UCS, Greedy) sul grafo, visualizzando i risultati e misurando le prestazioni:
 - Tempo di esecuzione
 - Utilizzo di memoria
 - Numero di operazioni eseguite
-

Monitoraggio delle Prestazioni

Per ciascun algoritmo, vengono misurati i seguenti parametri:

1. Tempo di esecuzione:

- Viene calcolato utilizzando la funzione `time.time()`:

```
start_time = time.time()
```

```
[Esecuzione dell'algoritmo]
```

```
end_time = time.time()
```

```
execution_time = end_time - start_time
```

2. Consumo di memoria:

- Monitorato tramite la libreria `tracemalloc`:

```
tracemalloc.start()
```

```
[Esecuzione dell'algoritmo]
```

```
peak_memory = tracemalloc.get_traced_memory()
```

```
tracemalloc.stop()
```

3. Numero di nodi esplorati:

- Ogni algoritmo tiene traccia del numero di nodi visitati e delle operazioni effettuate durante la ricerca.

Struttura delle Funzioni

Ecco una panoramica delle funzioni principali utilizzate nel progetto:

1. **parse_grafo(file_path)**
 - Legge i dati da un file e costruisce un dizionario (come lista di adiacenza) che rappresenta il grafo.
2. **traccia_percorso (genitori, start, goal)**
 - Ricostruisce il percorso a partire da un dizionario di genitori.
3. **bfs (grafo, start, goal)**
 - Implementazione della ricerca in ampiezza (Breadth-First Search).
4. **dfs (grafo, start, goal)**
 - Implementazione della ricerca in profondità (Depth-First Search).
5. **a_star (grafo, start, goal)**
 - Implementazione dell'algoritmo A*.
6. **uniform_cost_search(grafo, start, goal)**
 - Implementazione della ricerca a costo uniforme.
7. **greedy_search(grafo, start, goal, heuristic)**
 - Implementazione della ricerca Greedy, basata su una funzione euristica.
8. **esegui_ricerca_algoritmoX(grafo, start, goal, algoritmo)**
 - Funzione principale per eseguire un algoritmo di ricerca e monitorare le sue prestazioni.

Strumenti Utilizzati

- **Python:** Il progetto è sviluppato in Python, sfruttando librerie standard per la gestione del tempo (time), della memoria (tracemalloc) e della gestione di strutture dati (code con priorità tramite heapq).
 - **Tkinter:** Utilizzato per la GUI (l'interfaccia grafica), che permette di caricare il file del grafo, selezionare un algoritmo di ricerca e visualizzare i risultati.
-

Considerazioni Finali

Questo progetto permette di confrontare visivamente e quantitativamente le prestazioni di diversi algoritmi di ricerca, evidenziando differenze in termini di tempo, memoria e numero di operazioni. L'applicazione pratica su un dataset reale come la rete di coacquisti di Amazon fornisce un contesto concreto per valutare i vari algoritmi su grafi di grandi dimensioni.

Pseudocodice Funzioni del Progetto

Funzione parse_grafo (file_path):

- Aprire il file dal percorso specificato (file_path).
- Inizializzare un dizionario vuoto per rappresentare il grafo.
- Per ogni linea nel file:
 - Separare i nodi i e j (origine e destinazione).
 - Aggiungere j alla lista dei nodi adiacenti di i nel dizionario.
- Restituire il grafo sotto forma di lista di adiacenza (dizionario).

Funzione traccia_percorso (genitori, start, goal):

- Inizializzare una lista vuota per il percorso.
- Impostare current = goal.
- Ripetere finché current non è start:
 - Aggiungere current alla lista percorso.
 - Impostare current come genitori [current].
- Aggiungere start alla lista percorso.
- Restituire il percorso invertito (dal goal allo start).

Funzione bfs (grafo, start, goal):

Inizializzare una coda vuota e aggiungere il nodo start.

Inizializzare un set vuoto per tracciare i nodi visitati.

Inizializzare un dizionario vuoto per i genitori.

Ripetere finché la coda non è vuota:

 Estrai il nodo corrente dalla coda.

 Se il nodo corrente è goal, termina e ricostruisci il percorso.

 Per ogni nodo vicino al nodo corrente

 Se non è stato visitato:

 Aggiungerlo alla coda.

 Aggiornare il genitore del nodo.

 Segnare il nodo come visitato.

Se la coda si esaurisce senza trovare il goal, restituire fallimento.

Funzione dfs (grafo, start, goal):

Inizializzare uno stack vuoto e aggiungere il nodo start.

Inizializzare un set vuoto per tracciare i nodi visitati.

Inizializzare un dizionario vuoto per i genitori.

Ripetere finché lo stack non è vuoto:

 Estrai il nodo corrente dallo stack.

 Se il nodo corrente è goal, termina e ricostruisci il percorso.

 Per ogni nodo vicino al nodo corrente:

 Se non è stato visitato:

 Aggiungerlo allo stack.

 Aggiornare il genitore del nodo.

 Segnare il nodo come visitato.

Se lo stack si esaurisce senza trovare il goal, restituire fallimento.

Funzione a_star (grafo, start, goal, h):

Inizializzare una coda con priorità vuota e aggiungere il nodo start.

Inizializzare un dizionario per i costi, con start a 0.

Inizializzare un dizionario vuoto per i genitori.

Ripetere finché la coda non è vuota:

 Estrai il nodo corrente con il costo minore dalla coda.

 Se il nodo corrente è goal, termina e ricostruisci il percorso.

 Per ogni nodo vicino al nodo corrente:

 Calcolare il costo del percorso fino a quel nodo.

 Se è il percorso più economico trovato finora:

 Aggiorna il costo.

 Aggiungere il nodo alla coda con priorità.

 Aggiornare il genitore del nodo.

Se la coda si esaurisce senza trovare il goal, restituire fallimento.

Funzione uniform_cost_search (grafo, start, goal):

Inizializzare una coda con priorità vuota e aggiungere il nodo start.

Inizializzare un dizionario per i costi, con start a 0.

Inizializzare un dizionario vuoto per i genitori.

Ripetere finché la coda non è vuota:

 Estrai il nodo corrente con il costo più basso dalla coda.

 Se il nodo corrente è goal, termina e ricostruisci il percorso.

 Per ogni nodo vicino al nodo corrente:

 Calcolare il costo del percorso fino a quel nodo.

 Se è il percorso più economico trovato finora:

 Aggiorna il costo.

 Aggiungere il nodo alla coda con priorità.

 Aggiornare il genitore del nodo.

Se la coda si esaurisce senza trovare il goal, restituire fallimento.

Funzione greedy_search (grafo, start, goal, heuristic):

Inizializzare una coda con priorità vuota e aggiungere il nodo start.

Inizializzare un dizionario vuoto per i genitori.

Ripetere finché la coda non è vuota:

 Estrai il nodo corrente con la stima euristica più bassa dalla coda.

 Se il nodo corrente è goal, termina e ricostruisci il percorso.

 Per ogni nodo vicino al nodo corrente:

 Se non è stato visitato:

 Calcolare l'euristica del nodo.

 Aggiungere il nodo alla coda con priorità.

 Aggiornare il genitore del nodo.

Se la coda si esaurisce senza trovare il goal, restituire fallimento.

Funzione esegui_ricerca_algoritmoX (grafo, start, goal, algoritmo):

Inizializzare l'algoritmo selezionato (BFS, DFS, A*, UCS, Greedy).

Eseguire il monitoraggio del tempo:

 Registrare il tempo di inizio.

 Eseguire l'algoritmo selezionato.

 Registrare il tempo di fine e calcolare il tempo di esecuzione.

Monitorare il consumo di memoria:

 Avviare tracemalloc.

 Eseguire l'algoritmo.

 Recuperare la memoria di picco utilizzata e fermare tracemalloc.

Restituire e visualizzare il percorso trovato, il tempo e la memoria usata.

TEST

Specifiche tecniche del calcolatore utilizzato per la fase di test:

Processore: Intel® Core™ i7-14700KF 3,40 GHz

Memoria: 32GB DDR5 4800 MT/s

Tipo sistema: Sistema operativo a 64bit con processore basato su x64

I Test effettuati sono stati i seguenti:

- Ricerca percorso dal nodo 0 al nodo 100.
- Ricerca percorso dal nodo 0 al nodo 50.000.
- Ricerca percorso dal nodo 0 al nodo 100.000.
- Ricerca percorso dal nodo 0 al nodo 200.000.

Test	Algoritmo	Tempo di esec. (ms)	Picco di memoria (KB)	Nodi esplorati
0-100	BFS	0,1	6,23	4
0-100	DFS	44,4	3969,95	10993
0-100	A*	0,2	11,73	4
0-100	UCS	0,2	9,52	5
0-100	Greedy	0,2	7,55	41
0-50.000	BFS	54,0	5984,95	18
0-50.000	DFS	178,6	17751,05	35548
0-50.000	A*	148,7	7140,90	18
0-50.000	UCS	188,1	6978,63	19
0-50.000	Greedy	6,7	269,65	1828
0-100.000	BFS	115,6	11937,91	21
0-100.000	DFS	251,7	23895,08	29334
0-100.000	A*	341,0	13973,34	21
0-100.000	UCS	446,3	13509,63	27
0-100.000	Greedy	190,2	5507,41	41372
0-200.000	BFS	60,5	5984,95	19
0-200.000	DFS	9,3	962,05	2881
0-200.000	A*	243,0	13973,34	19
0-200.000	UCS	309,2	13509,63	30
0-200.000	Greedy	805,0	17413,66	190861

Osservazioni:

1. Breadth-First Search (BFS)

- Ottimalità del Percorso: BFS risulta **trovare sempre il percorso ottimo/più breve** (in termini di numero di passi) su grafi non pesati, come evidenziato dal numero costantemente basso di nodi esplorati.
- Tempo di Esecuzione: Generalmente **estremamente rapido**. La sua velocità è notevole, soprattutto considerando che garantisce l'ottimalità della soluzione trovata.

- Picco di Memoria: Possiede **picchi di memoria relativamente elevati** rispetto ad altri algoritmi come DFS, poiché esplora i livelli del grafo in ampiezza e deve mantenere in coda un numero potenzialmente elevato di nodi. La sua efficienza nel trovare soluzioni brevi contribuisce a contenere il tempo, ma la memoria rimane un fattore limitante su grafi molto ampi o con soluzioni lontane.

2. Depth-First Search (DFS)

- Ottimalità del Percorso: DFS **tende a trovare soluzioni estremamente lunghe e non ottimali**, data la sua caratteristica di ricerca in profondità che non considera la brevità del percorso. Il numero di "Nodi esplorati" (nel percorso) è spesso molto elevato.
- Tempo di Esecuzione: Le sue performance temporali sono **molto imprevedibili**. Può essere sorprendentemente veloce se la soluzione si trova presto lungo un ramo profondo che esplora per primo (come nel caso 0-200.000 nodi nei tuoi test), ma può diventare **molto lento** se deve esplorare rami molto lunghi prima di trovare una soluzione. Dipende fortemente dalla struttura specifica del grafo e dalla posizione della soluzione.
- Picco di Memoria: Il suo consumo di memoria è anch'esso **altamente variabile**. Può essere molto contenuto se la soluzione si trova presto, permettendogli di non memorizzare un grande numero di stati. Tuttavia, può **consumare molta memoria** se è costretto a esplorare rami molto lunghi e profondi prima di trovare una soluzione, accumulando nodi nello stack di ricorsione.

3. A*

- Ottimalità del Percorso: A* **trova sempre il percorso ottimo/a costo minimo** (se l'euristica è ammissibile e consistente), come confermato dal numero costantemente basso di "Nodi esplorati" che è paragonabile a quello di BFS.
- Tempo di Esecuzione: Risulta **generalmente tra i più rapidi e consistenti**. La sua capacità di utilizzare un'euristica per guidare la ricerca lo rende estremamente efficiente nel minimizzare il numero di nodi effettivamente visitati durante il processo di ricerca.
- Picco di Memoria: Il consumo di memoria è **significativo**, spesso **comparabile o talvolta superiore al BFS** su problemi di dimensioni maggiori, a causa della necessità di mantenere i nodi nella coda di priorità (frontiera). Tuttavia, la sua intelligenza nella ricerca spesso gli permette di essere più efficiente in termini di memoria rispetto a un BFS cieco su grafi molto ampi con soluzioni distanti.

4. Uniform Cost Search (UCS)

- Ottimalità del Percorso: UCS **garantisce di trovare il percorso a costo minimo** (e quindi il percorso più breve se i costi degli archi sono unitari), come indicato dal numero costantemente basso di "Nodi esplorati" che è quasi ottimale.
- Tempo di Esecuzione: Tende ad essere *leggermente più lento di A e BFS** (sebbene ancora molto efficiente), a causa dell'overhead associato all'uso di una coda di priorità per gestire i costi dei cammini.
- Picco di Memoria: Possiede **picchi di memoria comparabili a quelli dell'algoritmo A***, dato che anche esso mantiene i nodi nella coda di priorità per assicurare l'esplorazione basata sul costo.

5. Greedy Search

- Ottimalità del Percorso: Il Greedy Search **non garantisce l'ottimalità del percorso** e tende a trovare **soluzioni estremamente lunghe e inefficienti** (come evidenziato dal numero elevatissimo e variabile di "Nodi esplorati"). Si limita a scegliere il prossimo nodo che sembra più promettente secondo l'euristica, senza considerare il costo cumulativo del percorso.
- Tempo di Esecuzione: I tempi di esecuzione sono **altamente variabili**. Può essere molto rapido in alcuni scenari (come nel caso 0-50.000 nodi) se l'euristica lo guida rapidamente a una soluzione, anche se lunga. Tuttavia, può diventare **il più lento in assoluto** se l'euristica lo porta in percorsi non promettenti che richiedono un'esplorazione estesa (come nel caso 0-200.000 nodi).
- Picco di Memoria: Anche il picco di memoria è **altamente variabile**. Può essere sorprendentemente basso in alcuni casi, ma diventa **molto elevato** quando l'algoritmo è costretto a esplorare un numero massiccio di nodi per trovare una soluzione (come nel caso 0-200.000 nodi), data la lunghezza della soluzione trovata.